

Pràctica 3: Dashboard de Monitorització d'un CPD

Oriol Iglesias Farrà
Marc García Marco
Dani Soldevila

Projectes_DANI S.
Pràctica 3

ÍNDEX:

DOCUMENT TÈCNIC.....	3
1. Objectiu de la pràctica.....	3
2. Explicació de conceptes clau.....	3
Classe, objecte i mètode.....	3
Relació amb l'objecte Serial.....	3
3. Esquema de connexions (Fritzing).....	4
4. Diagrama de flux del programa.....	5
5. Proves.....	6
6. Explicació del projecte a GitHub.....	7
README.md.....	7

DOCUMENT TÈCNIC

1. Objectiu de la pràctica

L'objectiu d'aquesta pràctica és convertir l'ESP32 en un servidor web capaç de monitoritzar una sala de servidors (CPD). El sistema ha de mostrar dades ambientals en temps real (temperatura i lluminositat), activar alertes visuals i físiques, i oferir una interfície web amb disseny fix i pàgines secundàries.

2. Explicació de conceptes clau

Classe, objecte i mètode

Una **classe** és un model que defineix propietats i comportaments.

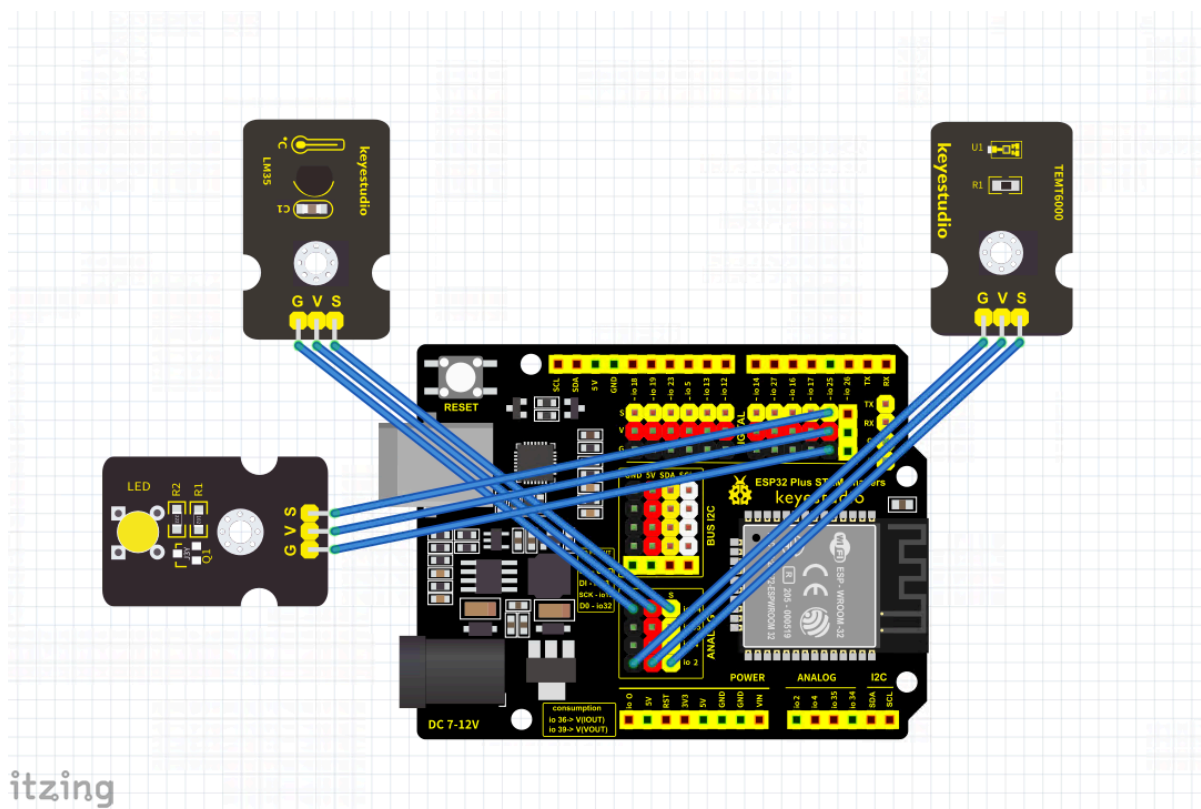
Un **objecte** és una instància concreta d'una classe.

Un **mètode** és una funció que pertany a una classe i defineix accions que pot fer l'objecte.

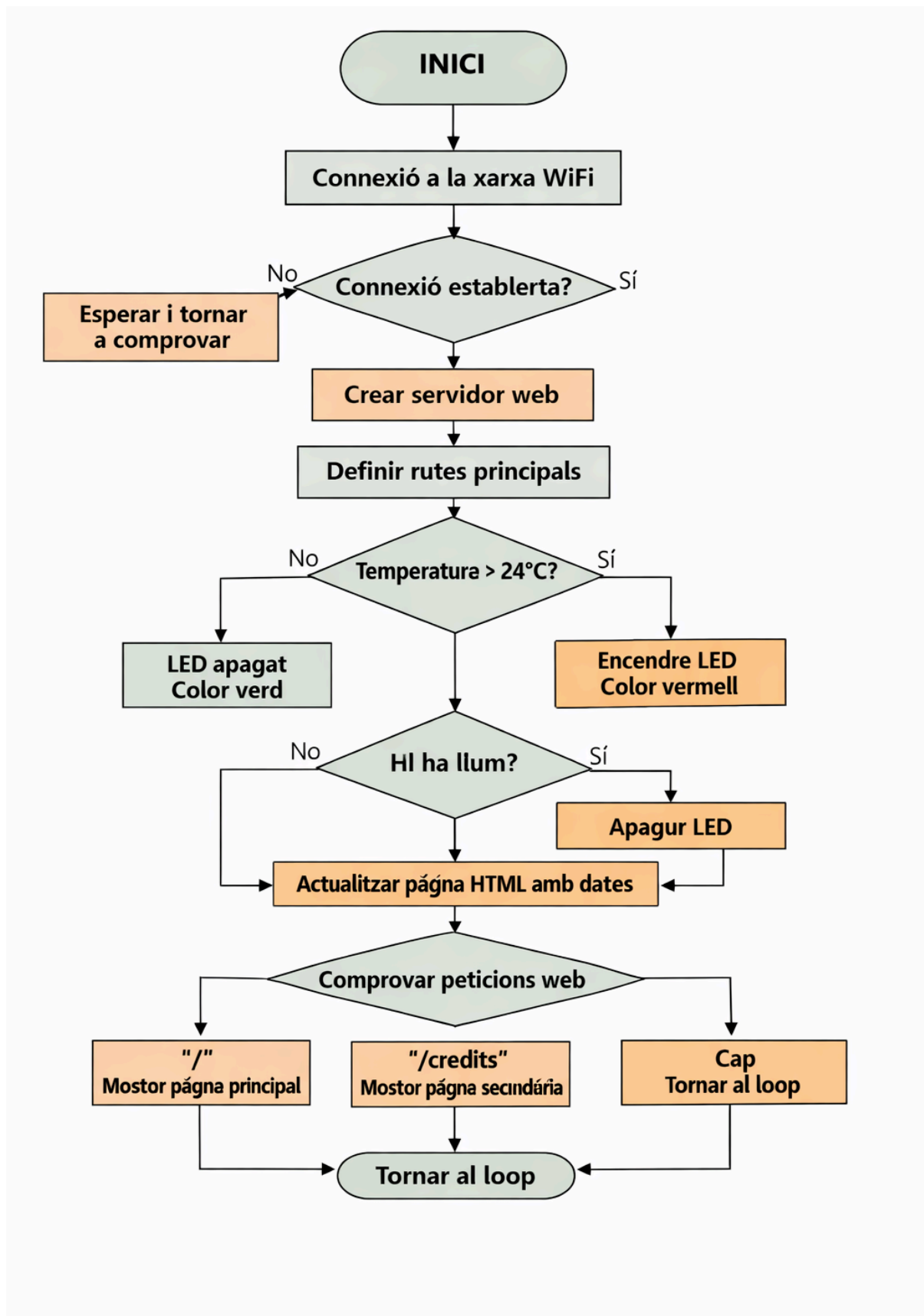
Relació amb l'objecte Serial

L'objecte **Serial** és una instància de la classe `HardwareSerial`. Permet comunicar l'ESP32 amb l'ordinador mitjançant el port USB, enviant i rebent dades textuais. En aquesta pràctica, s'utilitza per mostrar missatges de connexió i estat del servidor.

3. Esquema de connexions (Fritzing)



4. Diagrama de flux del programa



5. Proves

Durant les proves s'ha verificat que:

- L'ESP32 es connecta correctament a la xarxa WiFi i mostra la IP al monitor sèrie.
- La pàgina web mostra la temperatura i la llum en temps real.
- El color de la targeta canvia segons la temperatura (verd o vermell).
- El LED físic reacciona igual que la pàgina web.
- La pàgina secundària "/credits" funciona correctament.

Enllaç al vídeo demostratiu:

[ENLLAÇ DEL VÍDEO](#)

6. Codi

```
#include <WiFi.h>
#include <WebServer.h>

// =====
// CONFIGURACIÓN DE LAS CREDENCIALES WIFI
// =====
const char* STASSID = "TU_NOMBRE_DE_WIFI";
const char* STAPSK = "TU_CONTRASEÑA_DE_WIFI";

// =====
// ASIGNACIÓN DE PINES (Basado en Keyestudio GVS)
// =====
const int PIN_LED = 13;      // Pin digital para el módulo LED físico
const int PIN_TEMP = 34;     // Pin analógico para el sensor de temperatura LM35
                              (Analog)
const int PIN_LLUM = 35;     // Pin analógico para el sensor de luz TEMT6000 (Analog)

// Instancia del servidor web en el puerto 80
WebServer server(80);

// Variables globales para almacenar las lecturas del CPD
float temperatura = 0.0;
int lluminositat = 0;
String colorTargeta = "green"; // Controla el color de la tarjeta en la web (verd o vermell)

// =====
// LÓGICA DE CONTROL DE SENSORES Y ALERTAS
// =====
```

```

void actualitzarSensors() {
    // --- Lectura de Temperatura (LM35) ---
    int rawTemp = analogRead(PIN_TEMP);
    // Conversi3n anal3gica a voltios (ESP32: 12 bits = 4095, V_REF = 3.3V)
    // El LM35 entrega 10mV por cada grado Celsius
    float voltatgeTemp = (rawTemp / 4095.0) * 3.3;
    temperatura = voltatgeTemp * 100.0;

    // --- Lectura de Luminositat (TEMT6000) ---
    lluminositat = analogRead(PIN_LLUM);

    // --- Control de Alertas (Diagrama de flujo) ---
    // Condici3n 1: Temperatura > 24°C -> Encender LED f3sico y tarjeta roja en web
    if (temperatura > 24.0) {
        digitalWrite(PIN_LED, HIGH); // Encendre LED (Color vermell)
        colorTargeta = "red";
    }
    // Condici3n 2: Hay luz -> Apagar LED f3sico y tarjeta verde en web
    // Ajustad el umbral de luz (ej. 1500) seg3n la iluminaci3n ambiental detectada por el
    // TEMT6000
    else if (lluminositat > 1500) {
        digitalWrite(PIN_LED, LOW); // Apagar LED
        colorTargeta = "green"; // LED apagat / Color verd en web
    }
    // Si no se cumple lo anterior (temperatura normal y oscuridad), se mantiene
    // apagado/verde
    else {
        digitalWrite(PIN_LED, LOW);
        colorTargeta = "green";
    }
}

// =====
// MANEJADORES DEL SERVIDOR WEB (RUTAS)
// =====

// Ruta Principal ("/") - Muestra el Dashboard del CPD
void handleRoot() {
    actualitzarSensors(); // Actualizar datos antes de servir la p3gina

    String html = "<!DOCTYPE html><html lang='ca'>";
    html += "<head><meta charset='UTF-8'><meta name='viewport'";
    content="width=device-width, initial-scale=1.0">";
    html += "<title>Dashboard CPD</title>";
    html += "<style>";
    html += "body { font-family: Arial, sans-serif; background-color: #f4f4f9; text-align: center;";
    padding: 20px; }";

```

```

    html += ".container { max-width: 600px; margin: auto; background: white; padding: 20px;
border-radius: 10px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }";
    html += ".card { padding: 20px; margin: 15px 0; border-radius: 8px; color: white; font-size:
24px; font-weight: bold; transition: background 0.5s; }";
    html += ".green { background-color: #2ecc71; }"; // Color verde (estado normal)
    html += ".red { background-color: #e74c3c; }"; // Color rojo (alerta > 24°C)
    html += ".info-card { background-color: #34495e; padding: 20px; margin: 15px 0;
border-radius: 8px; color: white; font-size: 24px; font-weight: bold; }";
    html += "nav a { margin: 0 10px; text-decoration: none; color: #3498db; font-weight: bold; }";
    html += "</style>";
    // Auto-refresh cada 5 segundos para actualización en tiempo real
    html += "<script>setInterval(function(){ window.location.reload(); }, 5000);</script>";
    html += "</head><body>";

    html += "<div class='container'>";
    html += "<h1>Monitorització de Sala de Servidors (CPD)</h1>";
    html += "<nav><a href='/'>Dashboard</a> | <a href='/credits'>Crèdits</a></nav><br>";

    // Tarjeta Dinámica de Temperatura (Cambia de color según estado)
    html += "<div class='card " + colorTargeta + ">";
    html += "Temperatura: " + String(temperatura, 1) + " &deg;C";
    html += "</div>";

    // Tarjeta de Luminosidad
    html += "<div class='info-card'>";
    html += "Lluminositat: " + String(lluminositat) + " ADC";
    html += "</div>";

    html += "<p>Dades actualitzades en temps real de forma automàtica.</p>";
    html += "</div>";
    html += "</body></html>";

    server.send(200, "text/html", html);
}

// Ruta Secundaria ("/credits") - Información de los integrantes
void handleCredits() {
    String html = "<!DOCTYPE html><html lang='ca'>";
        html += "<head><meta charset='UTF-8'><meta name='viewport'
content='width=device-width, initial-scale=1.0'>";
    html += "<title>Crèdits del Projecte</title>";
    html += "<style>";
    html += "body { font-family: Arial, sans-serif; background-color: #f4f4f9; text-align: center;
padding: 20px; }";
    html += ".container { max-width: 600px; margin: auto; background: white; padding: 20px;
border-radius: 10px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }";
    html += "nav a { margin: 0 10px; text-decoration: none; color: #3498db; font-weight: bold; }";
    html += "ul { list-style-type: none; padding: 0; font-size: 18px; line-height: 2; }";

```



```

html += "</style></head><body>";

html += "<div class='container'>";
html += "<h1>Crèdits del Projecte</h1>";
html += "<nav><a href='/'>Dashboard</a> | <a href='/credits'>Crèdits</a></nav><br>";
html += "<h3>Desenvolupat per:</h3>";
html += "<ul>";
html += "<li><strong>Oriol Iglesias Farrà</strong></li>";
html += "<li><strong>Marc García Marco</strong></li>";
html += "</ul>";
html += "<p>Projectes - Monitorització d'un CPD</p>";
html += "</div>";
html += "</body></html>";

server.send(200, "text/html", html);
}

// =====
// CONFIGURACIÓN INICIAL (Setup)
// =====
void setup() {
  // Inicialización del puerto Serie
  Serial.begin(115200);
  delay(10);
  Serial.println("\nIniciant sistema de monitorització CPD...");

  // Configuración de pines de entrada/salida
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW); // Asegurar que el LED empiece apagado

  // Conexión a la red WiFi (Bucle del diagrama de flujo)
  Serial.print("Connectant a la xarxa: ");
  Serial.println(STASSID);
  WiFi.begin(STASSID, STAPSK);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print("."); // Esperar i tornar a comprovar
  }

  // Conexión establecida con éxito
  Serial.println("\nConnexió WiFi establerta amb èxit!");
  Serial.print("Adreça IP assignada: ");
  Serial.println(WiFi.localIP()); // Muestra la IP en el monitor serie

  // Definición de las rutas del servidor web
  server.on("/", handleRoot);
  server.on("/credits", handleCredits);

```

```
// Iniciar el servidor
server.begin();
Serial.println("Servidor web HTTP iniciat.");
}

// =====
// BUCLE PRINCIPAL (Loop)
// =====
void loop() {
  // Revisar y procesar las peticiones de los clientes web conectados
  server.handleClient();

  // Pequeño delay de cortesía para no saturar el procesador de la ESP32
  delay(10);
}
```

7. Explicació del projecte a GitHub

El repositori s'ha creat amb el nom **practica3-dashboard-cpd** i conté les carpetes següents:

- **/src** → codi font (main.ino, .cpp, .h)
- **/doc** → documentació, imatges i vídeo

S'ha creat un **Project Kanban** amb les columnes *To do*, *Doing* i *Done*, filtrant per l'etiqueta **practica3**.

Cada tasca s'ha definit com un *issue* independent. Quan una tasca s'ha completat, s'ha fet un commit amb el missatge:

Closes #número_issue

Per exemple:

"Afegit codi HTML per la pàgina principal — Closes #3"

Enllaç al repositori GitHub:

<https://github.com/marcusytespro/iot-esp32-smx.git>

README.md

Pràctica 3 – Dashboard de monitorització d'un CPD

Objectiu

Convertir l'ESP32 en un servidor web que mostri dades ambientals en temps real i activi alertes visuals i físiques.

Contingut del repositori

- /src → Codi font (main.ino, .cpp, .h)
- /doc → Documentació, imatges, PDF i vídeo
- README.md → Explicació general